



Samueli
School of Engineering

CS-M151B

Computer Systems Architecture

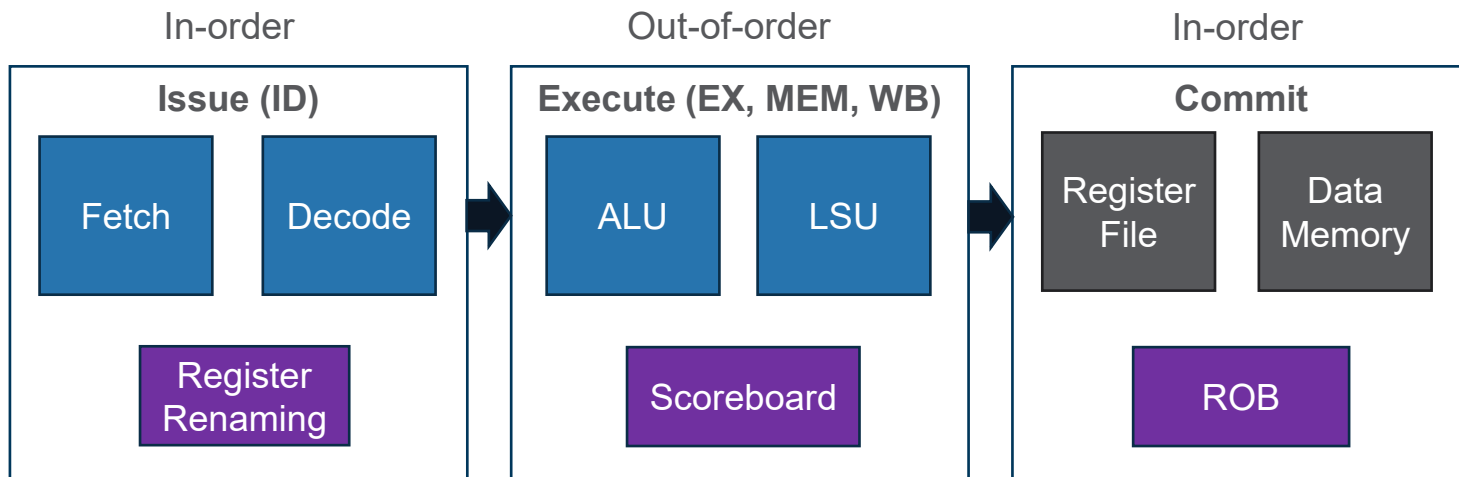
Blaise Tine
UCLA Computer Science

Agenda

- Project #3
- Out-of-order execution
- Exceptions

Recap: Out of order CPU

- + **Renaming:** WAR + WAW
- + **Scoreboarding:** RAW
- + **Reorder-Buffer:** In-order commit



Recap: Loads and Stores

Queue LD/ST instructions

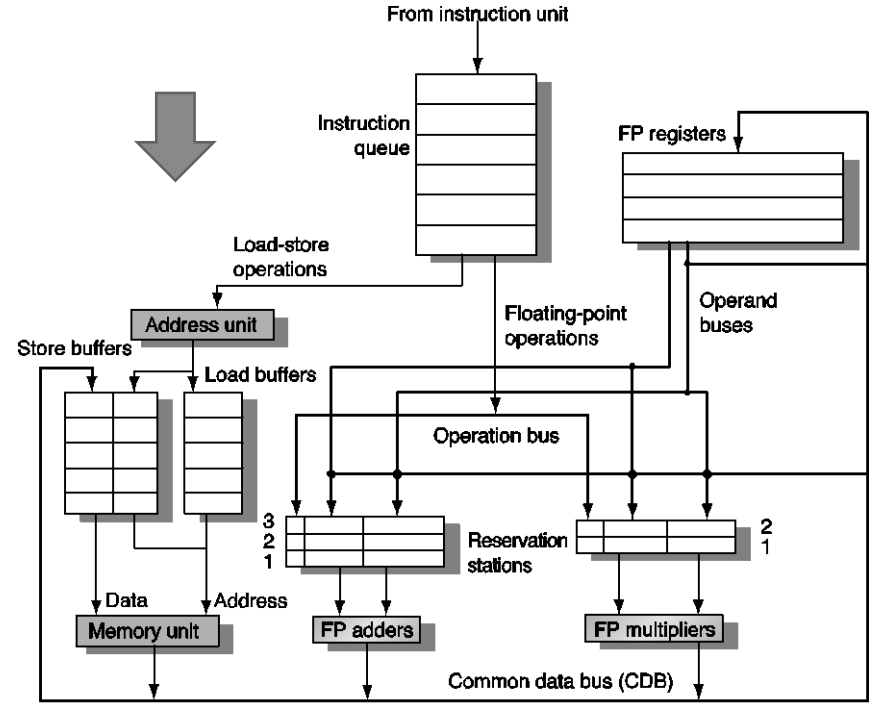
- Similar to ROB allocation

Source operands dependency

- Similar to RS allocation
- Use CDB

Enforce in-order memory update

- Use ROB



Recap: Unified Reservation Stations

RS (Adder)

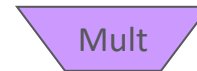
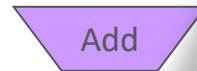


RS (Mul/Div)



RS (Unified)

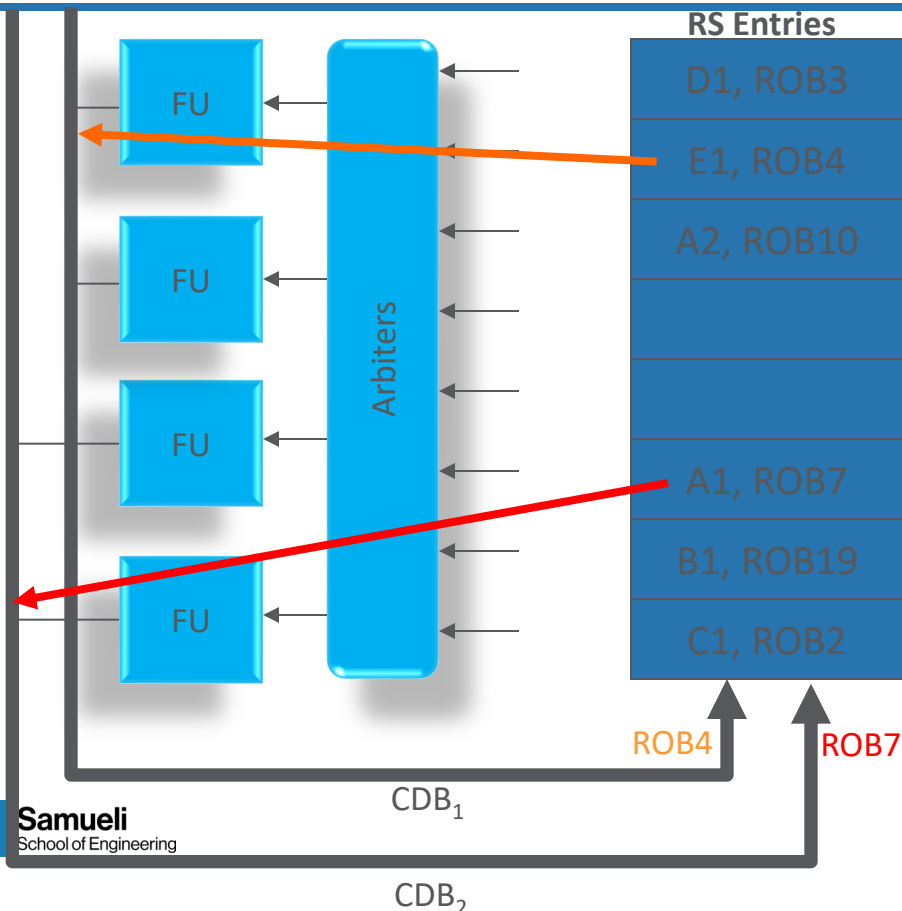
Can hold 5 adds



Or 5 multiplies

Or any combination

Recap: Multiple CDBs

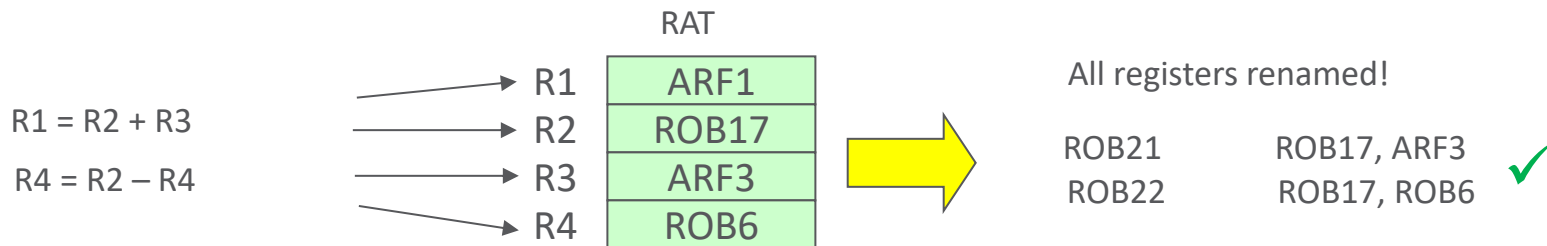


- Reduce write contention
- But there's a cost
 - each RS entry must compare each source operand with each CDB
 - more logic => more area => more power => \$\$\$
- We do this today in CPUs!

Dual-Issue Super-scalar

- Need to check resource availability for two instructions (RS's/ROB entries)
 - Depending on resources, may issue 0,1 or 2
- Read RAT/ARF/ROB for operands
 - Renaming is a little trickier (next slide)
- Update RS's/ROB entries (not too hard)

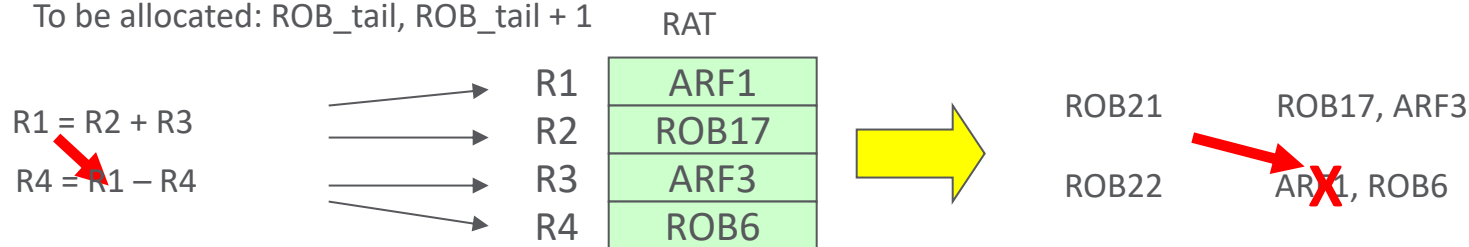
Dual-Rename



Read current mappings for all operands

New destinations are just in the next two ROB entries

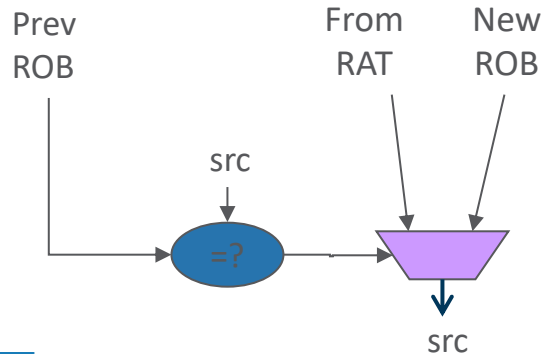
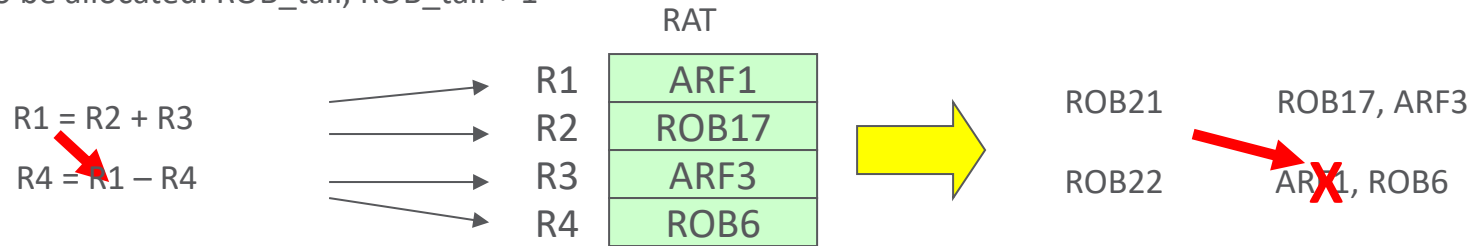
To be allocated: ROB_tail, ROB_tail + 1



Dual-Rename (2)

New destinations are just in the next two ROB entries

To be allocated: ROB_tail , $\text{ROB_tail} + 1$



Need to check for RAW dependencies within your issue window

Supporting Branch Prediction

- We speculate on branch target
- Resume execution on speculated PC
- If successful => we just continue
- If mispredicted => we flush
- How do we restore pipeline state?

Add r1, r2, r3

Br target

Sub r3, r5, r6

Mul r2, r5, #1

Supporting Branch Prediction

- Handling Pipeline Flushes:
 1. Take snapshot of current state
 2. Speculate
 3. if (misspredicted)
 4. Flush “younger” pending instructions
 5. Flush pipeline buffers
 6. Restore snapshot
 7. else:
 8. Release snapshot resource
 9. Resume

Add r1, r2, r3

Br target

Sub r3,r5, r6

Mul r2, r5, #1

Supporting Branch Prediction

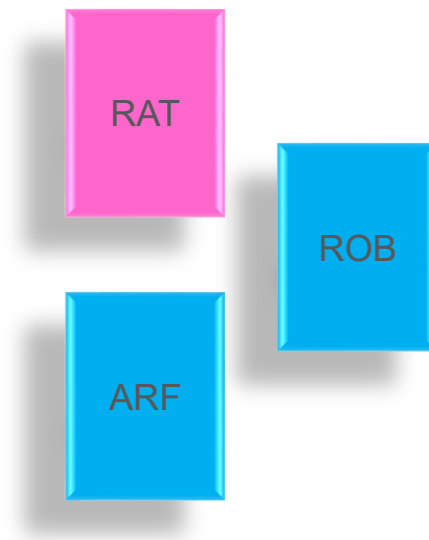
- Handling Pipeline Flushes:

1. Take snapshot of current state – how?
2. Speculate
3. if (misspredicted)
4. Flush “younger” pending instructions – how?
5. Restore snapshot
6. else:
7. Release snapshot resource
8. Resume

Add r1, r2, r3
Br target
Sub r3,r5, r6
Mul r2, r5, #1

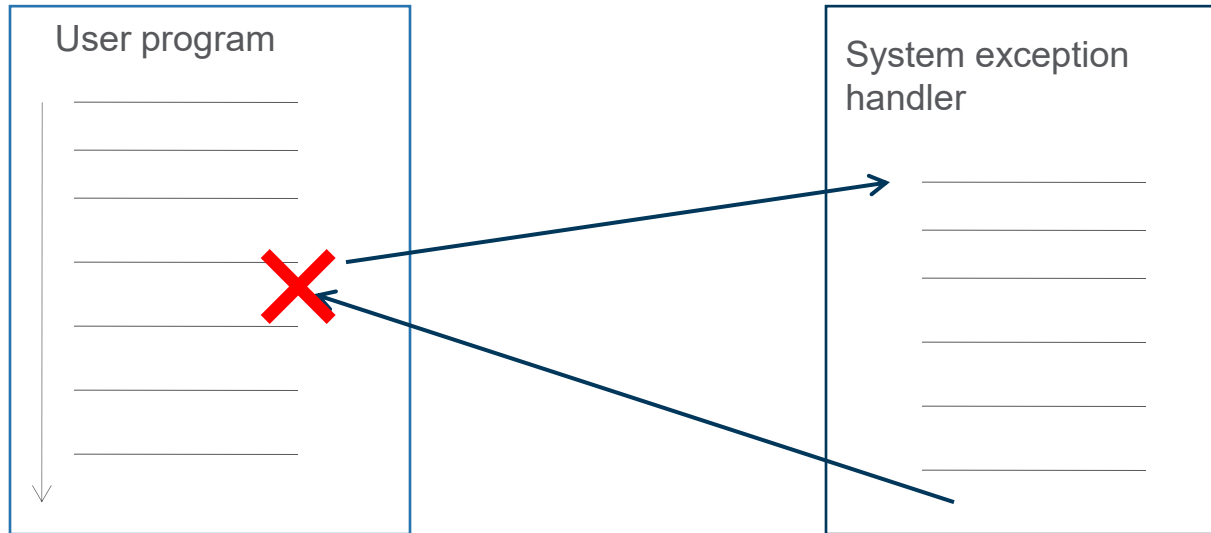
Speculation and Check Point

- Use the RAT as checkpoint snapshot
- Can support multiple checkpoints
- Use ROB index to identify checkpoint
- Use ROB index to flush “younger” instructions
 - Use ROB index as age
 - Compare instruction age
 - Flush pipeline queues + RSs + FUs + ROB
 - ROB index overflow?



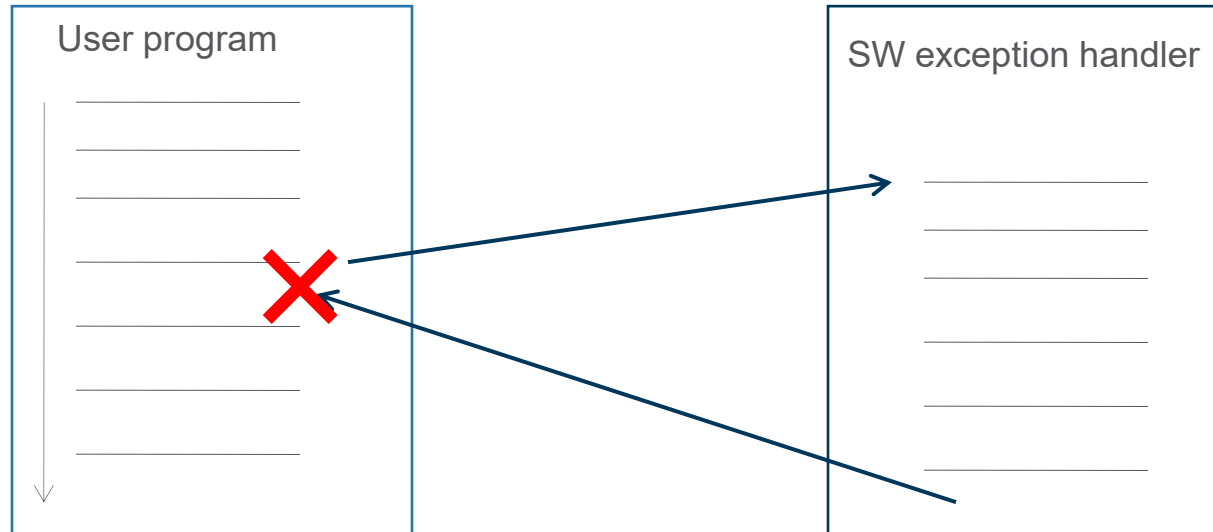
Exceptions

- What is exception?
 - Unusual or unexpected program event.
 - A system service takes action to handle the event



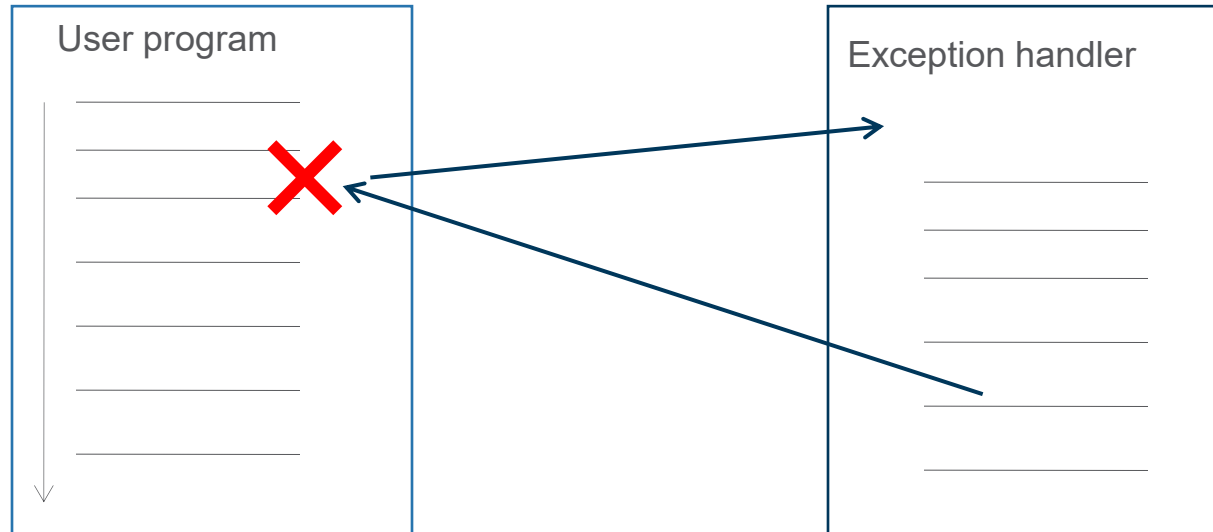
S/W Exceptions

- Software exceptions are user-level unusual events
 - Defined by the programming language



S/W Exceptions

- Software exceptions triggers
 - Hardware: *try { z = y / x; } catch*
 - Software: *if (x == 0) => throw*



Interrupts vs Exceptions

- Interrupts
 - Cased by external events
 - e.g. network, keyboard, timer
 - **Asynchronous** to program execution
 - May be handled between instructions
 - Simply suspend and resume user program

Interrupts vs Exceptions

- Exceptions
 - Caused by internal events
 - **Synchronous** to program execution
 - Condition must be remedied by the handler
 - Program may resume or abort

Question

- Which RISC-V instructions can execute without exceptions?

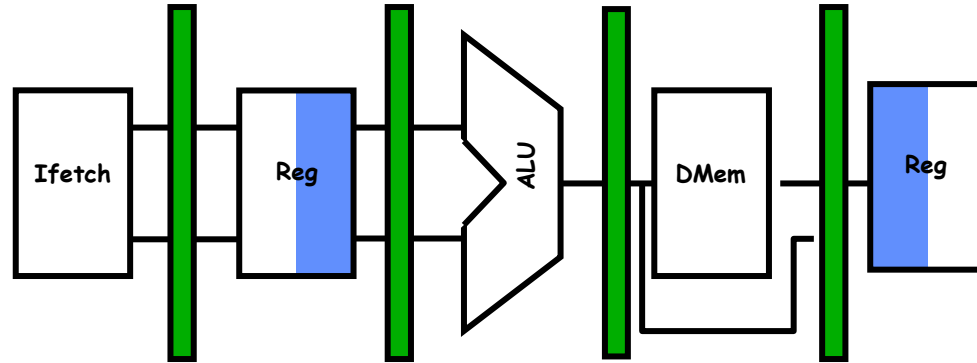
Types of Exceptions

- Arithmetic
 - Overflow, Underflow
 - Division by zero
- Faults
 - Page fault
 - Invalid opcode
- Errors
 - Memory parity bits
- Traps
 - Breakpoint

Precise Exceptions

- What supporting precise exceptions entails?
 - All previous instructions **have been executed**.
 - All following instructions **have not been modified** the process state
 - The faulting instruction **may or may not** have been executed, depending on the definition of the architecture and the cause of the exception.

Exceptions in in-order pipeline



Load

Div

Load

Div

Load

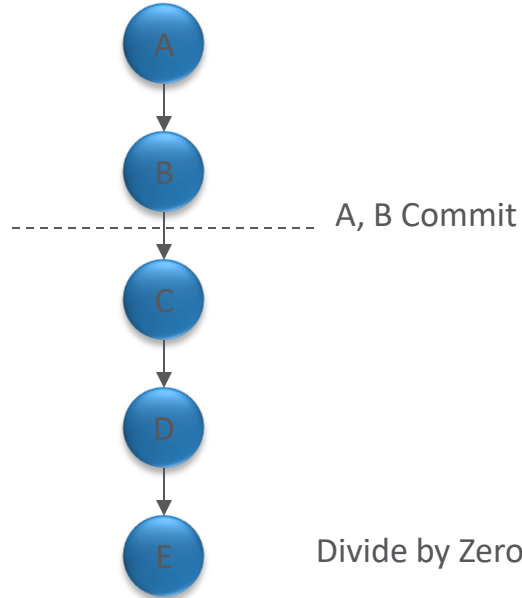


Load

Load

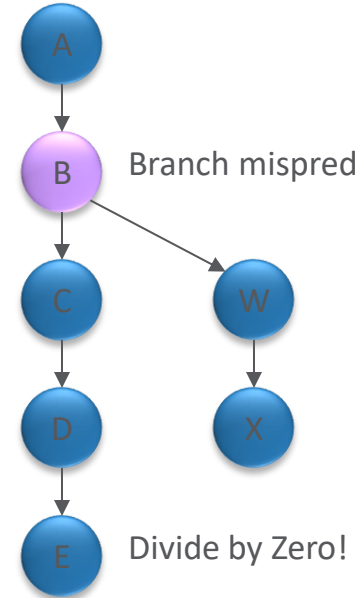
Exceptions in out-of-order pipeline

- What happens if a speculatively executed instruction faults?



Outside world sees: A, B, fault!

Should have been: A, B, C, D, fault!



Fault should never be seen!

Solution: handle faults like other results

- ROB commits instructions to register file when
 - It is oldest for in-order state update
 - It is known to be non-speculative
- Do the same with faults!
 - At exec, make note of any faults, but don't expose to outside world
 - If the instruction gets to commit, then expose the fault

Solution: handle faults like other results

- Handling faults at ROB commit time
 - No need to restore checkpoint
 - We are already at the top,
 - “Older” instructions have committed already
 - Only “younger” instructions remain
 - Flush the whole ROB
 - Flush pipeline queues + RSs + Fus
 - Flush the RAT
 - Update CSRs and redirect PC to Exception handler

Attendance

- Please Fill the form with today's keyword to register your attendance.



Cache

Memory Latency is Long

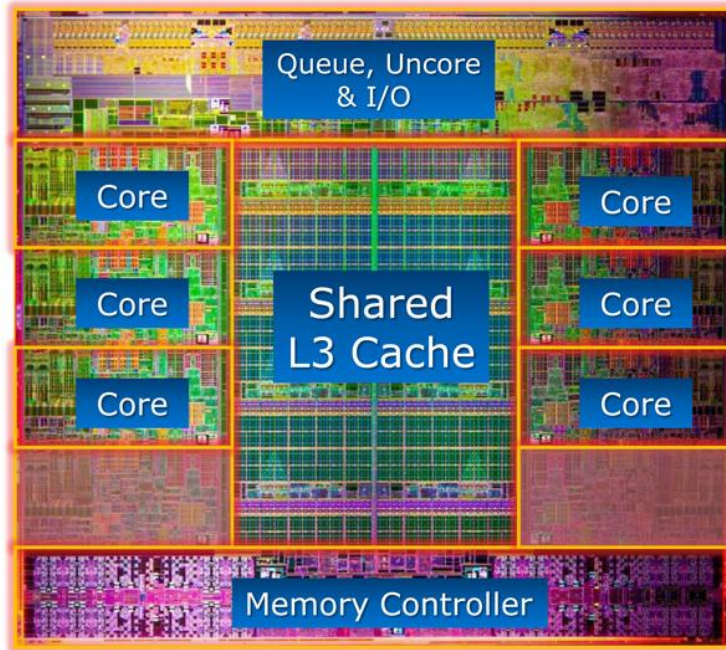
- 60-100ns is common
- What this really means for the CPU in term of clock cycles?
 - Assume 2 GHz CPU
 - Clock cycle time? = 0.5ns
 - 100ns memory → 200 cycle memory latency!

Memory Latency is Long

- Why is accessing memory slow?
 - Storage technology
 - DRAM is slower than SRAM (row buffer refreshes, DRAM cells)
 - Physical distance
 - Usually outside the core, sharing of the memory by multiple cores and peripherals (hard-drive)
 - Bus bandwidth
 - Growing number of cores saturate the memory bus

Memory Latency is Long

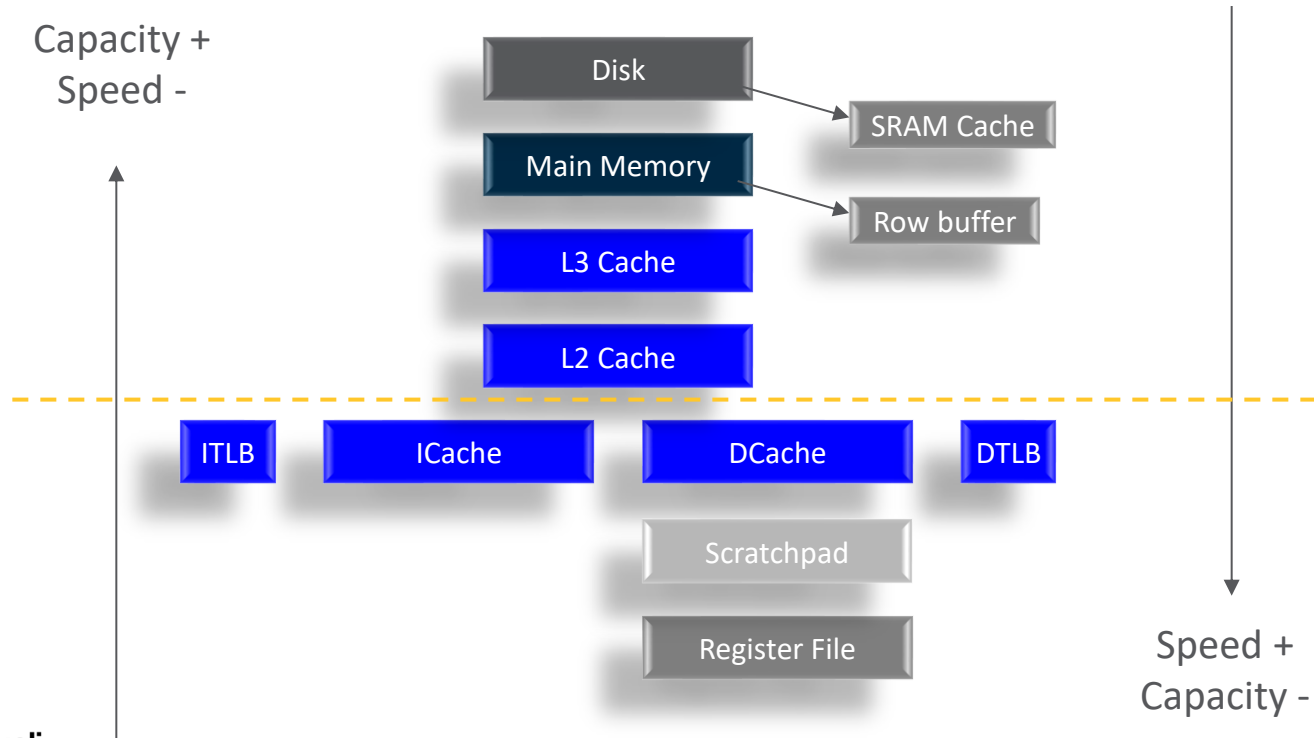
- Solution: hardware caches



Intel Sandy Bridge (2011)

- 6x L1-D: 64 KB
- 6x L1-I: 64 KB
- 6x L2: 256 KB
- 1x L3: 20,480 KB (20 MB)

Storage Hierarchy and Locality



Data Locality

- Spatial Locality
 - If data item needed now, nearby data likely to be needed in near future
- Temporal Locality
 - If data item needed now, it is likely to be needed again in near future

Locality with caches

- Exploiting spatial locality
 - Also bring nearby data close
 - Block size
 - Cache size

Locality with caches

- Exploiting temporal locality
 - Keep recently used data in fast memory close to the processor
 - Hierarchy
 - Replacement policies

Review Questions

Memory addresses A , $A+1$, $A+2$, $A+3$, $A+4$

- Spatial locality or temporal locality?

Review Questions

Memory addresses A , $A+1$, $A+2$, $A+3$, $A+4$

- Spatial locality or temporal locality?
- **Spatial**

Review Questions

Memory addresses A,B,C,A,B,C,A,B,C

- Spatial locality or temporal locality?

Review Questions

Memory addresses A,B,C,A,B,C,A,B,C

- Spatial locality or temporal locality?
- Temporal

Cache Basics

Fast (but small) memory close to processor

When data referenced

- If in cache, use cache instead of memory
- If not in cache, bring into cache
(actually, bring entire **block** of data, too)
- Maybe have to kick something else out to do it!

Key: Optimize the
average memory
access latency

Cache Basics

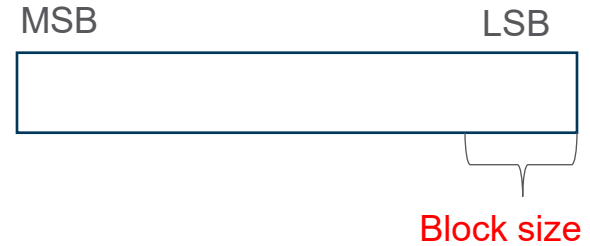
Important decisions

- **Placement:** where in the cache can a block go?
- **Identification:** how do we find a block in cache?
- **Replacement:** what to kick out to make room in cache?
- **Write policy:** what do we do about stores?

Cache Basics

Cache consists of block-sized **lines**

- Line size typically power of two
 - Why?
- Typically 16 to 128 bytes in size
 - Why?



Cache Basics

Example

- Suppose cache block size is 128 bytes
- Read data at address $A=0x7ffa3f4$
- What is the block base address in bytes?

Cache Basics

Example

- Suppose cache block size is 128 bytes
- Read data at address $A=0x7ffa3f4$
- What is the block base address in bytes?
- **Answer: $0x7ffa380$**

Cache Basics

Example

- Suppose cache block size is 64 bytes
- Lowest 6 bits determine **offset** within block
- What do we use the offset for?

Whiteboard

Cache Basics

Example

- Suppose cache block size is 64 bytes
- Lowest 6 bits determine **offset** within block
- What do we use the offset for?
- **Answer: Access to CPU data (e.g. 4 bytes - 32-bit)**

Cache Placement

Placement

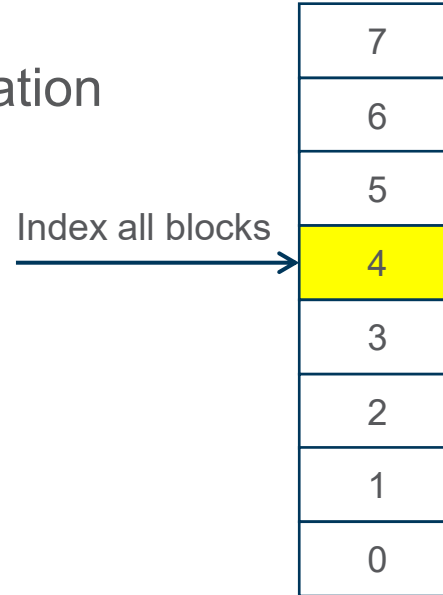
- Where to place a memory block in cache lines?

Placement Policies

- **Direct mapped** (block can go to only one line)
 - Index the cache using memory address
- **Fully Associative** (block can go to any line)
 - Lookup the cache using tag
- **Set-associative** (block can go to one of N lines)
 - Mixing indexing and lookup

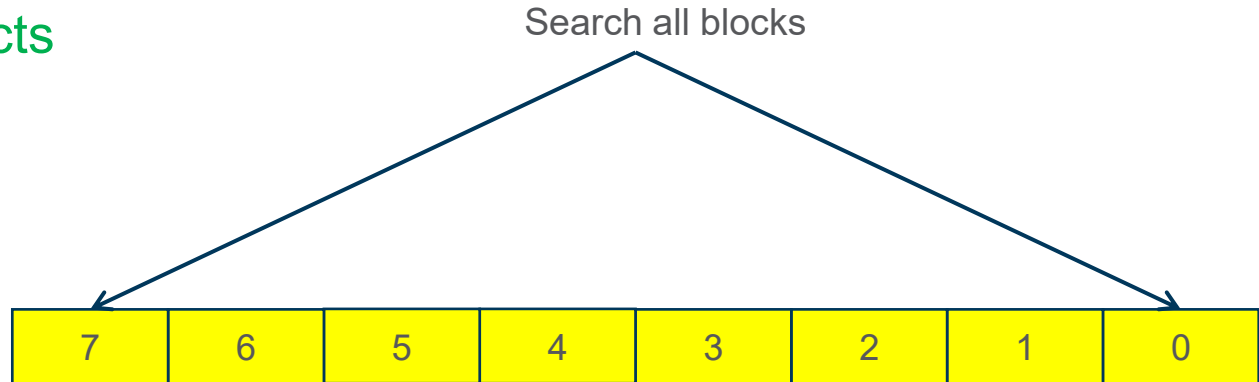
Cache Placement

- Direct Mapped
 - Fast access (indexing)
 - Block A can only be stored in one location
 - Compare address with one tag
 - No search
 - May have block conflicts



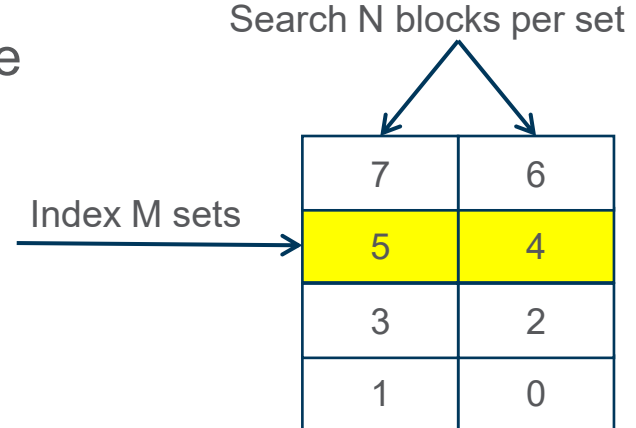
Cache Placement

- Fully Associative
 - Slow access (lookup)
 - Compare address with each tag
 - Block A can be stored in any cache line
 - Expensive search
 - No block conflicts



Cache Placement

- Set Associative
 - Average access (indexing + **lookup**)
 - Compare address with N tags
 - Block A can be stored in exactly one set but in any cache line in the set
 - **Less block conflicts**



Cache Identification

When address referenced, need to

- Find whether its data is in the cache
- If it is, find where in the cache
- This is called a cache **lookup**

Each cache line must have

- A **valid** bit (1 if line has data, 0 if line empty)
 - We also say the cache line is valid or invalid
- A **tag** to identify which block is in the line

Cache Address



Offset: Byte offset in block

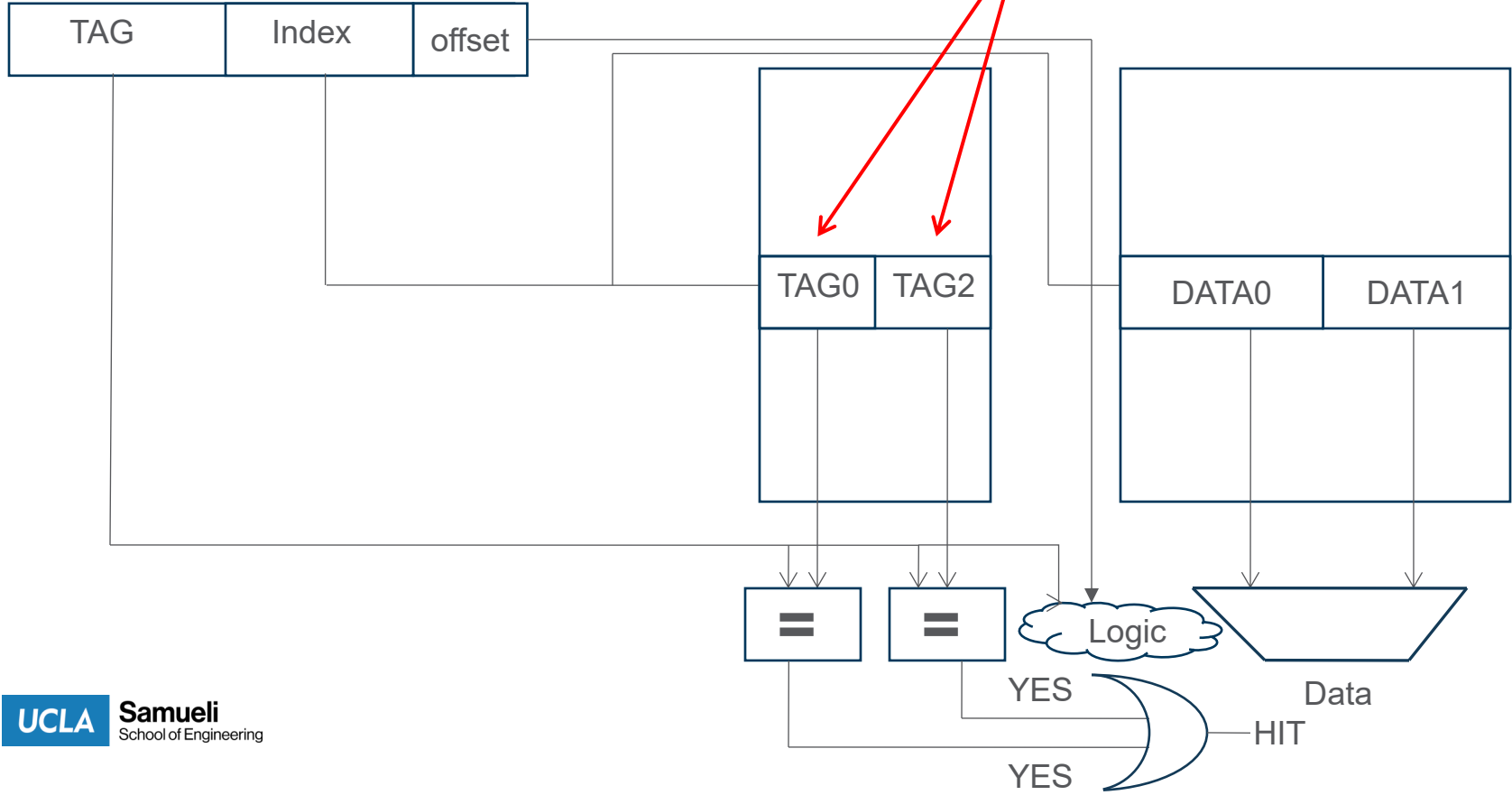
Index: Which set in the cache is the block located

Tag: Need to match address tag in cache

Set: Group of blocks corresponding to same index

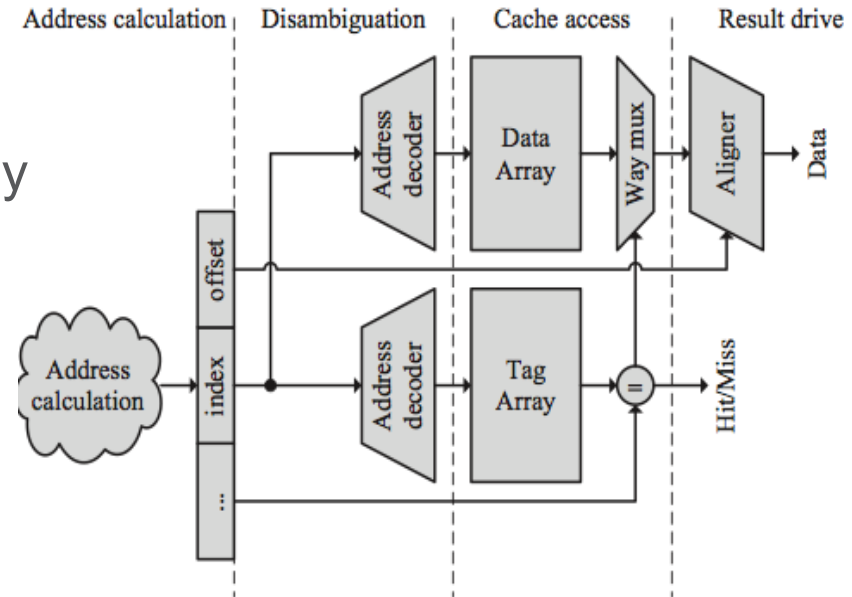
Review: Cache Indexing

2 Ways set associative



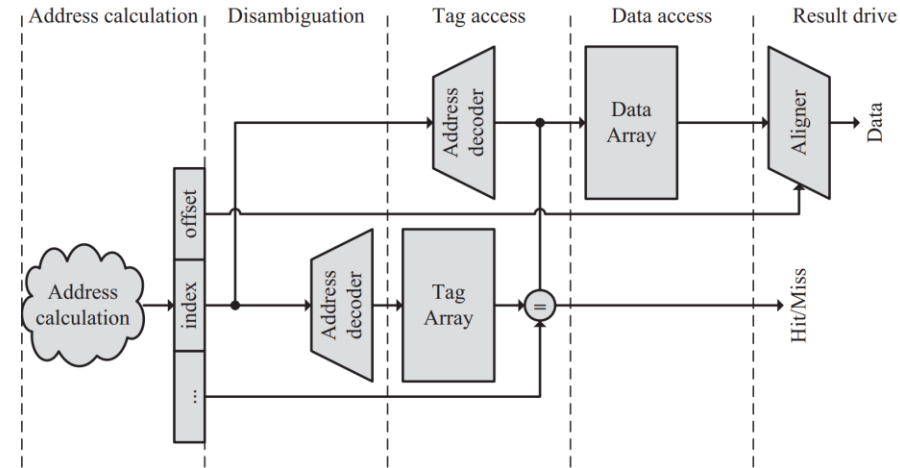
Parallel TAG and Data Array Access Pipeline

- Four pipeline stages
- Tag & Data parallel access
- Need a way multiplexer
 - Fetch ways lines from data array
 - Since we don't have the way index yet from tag compare
- Two critical paths
 - Tag access + compare
 - Data access + compare + mux

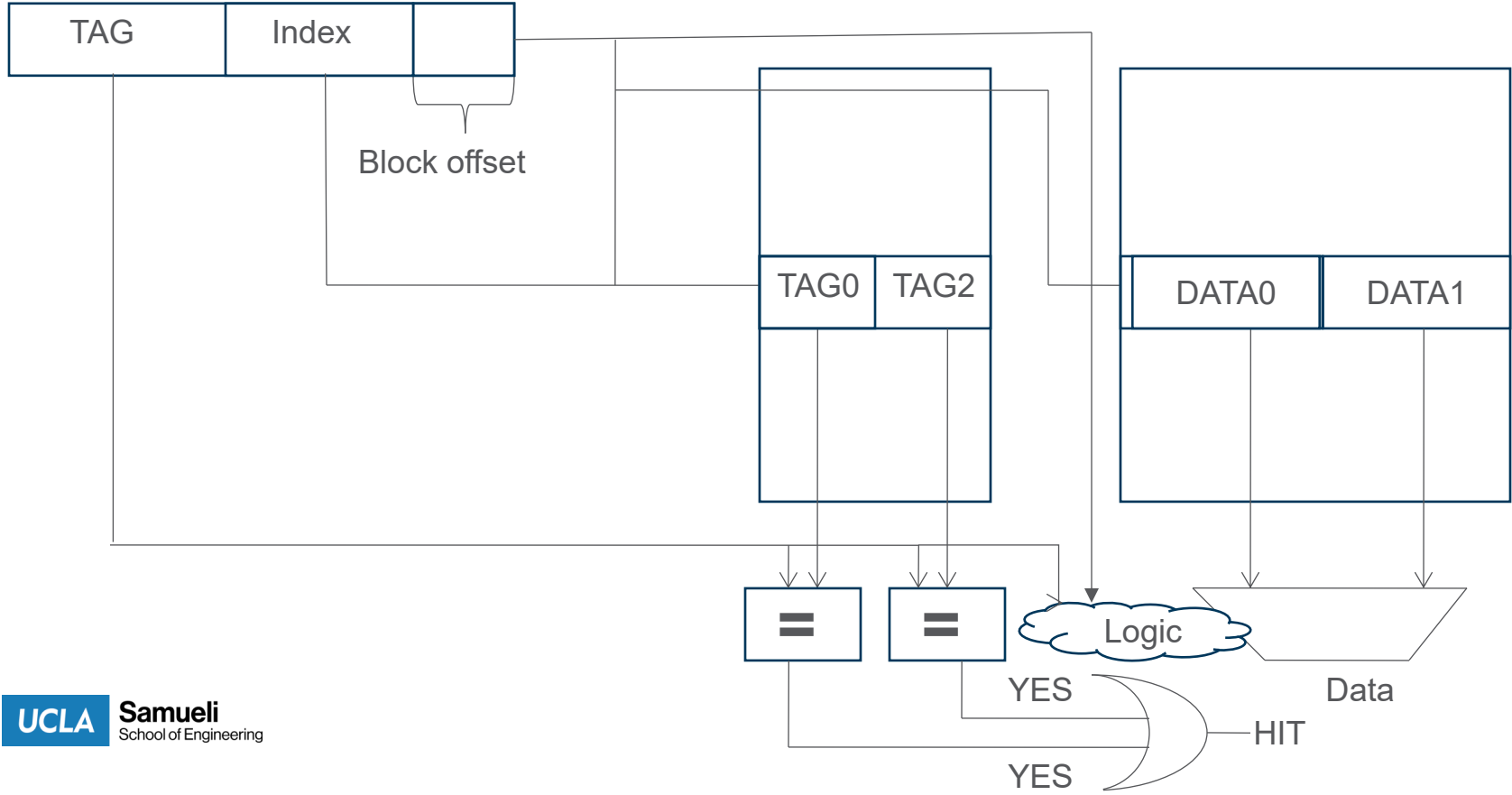


Serial TAG and Data Array Access Pipeline

- Five pipeline stages
 - One extra cycle
- Tag & Data serial access
- No way multiplexer
 - We have the way index
 - We can select our data line
- One smaller critical path
 - Tag access + compare



Cache Indexing



Q&A

Acknowledgement

- This course is partly inspired and use materials from the following courses created by my colleagues:
 - ECE-M1126C: Nader Sehatbakhsh @ UCLA
 - CS3220: Hyesoon Kim @ gatech